

LAB EXERCISE - 4

Design and Simulate Horizontal Database Sharding using MongoDB

Team Members

S.No	Name	Register Number
1	Dinesh Kumar J	3122245001042
2	Gopinath S	3122245001048
3	Harish R	3122245001055
4	Srikrishna O S	3122245001312

1) AIM

To deploy a MongoDB sharded cluster using Docker Compose, configure horizontal database sharding for the College database, distribute the Student collection across multiple shard servers using RollNo as the shard key, and analyze the scalability and query performance of a distributed database system.

2) SOFTWARES INSTALLED AND USED

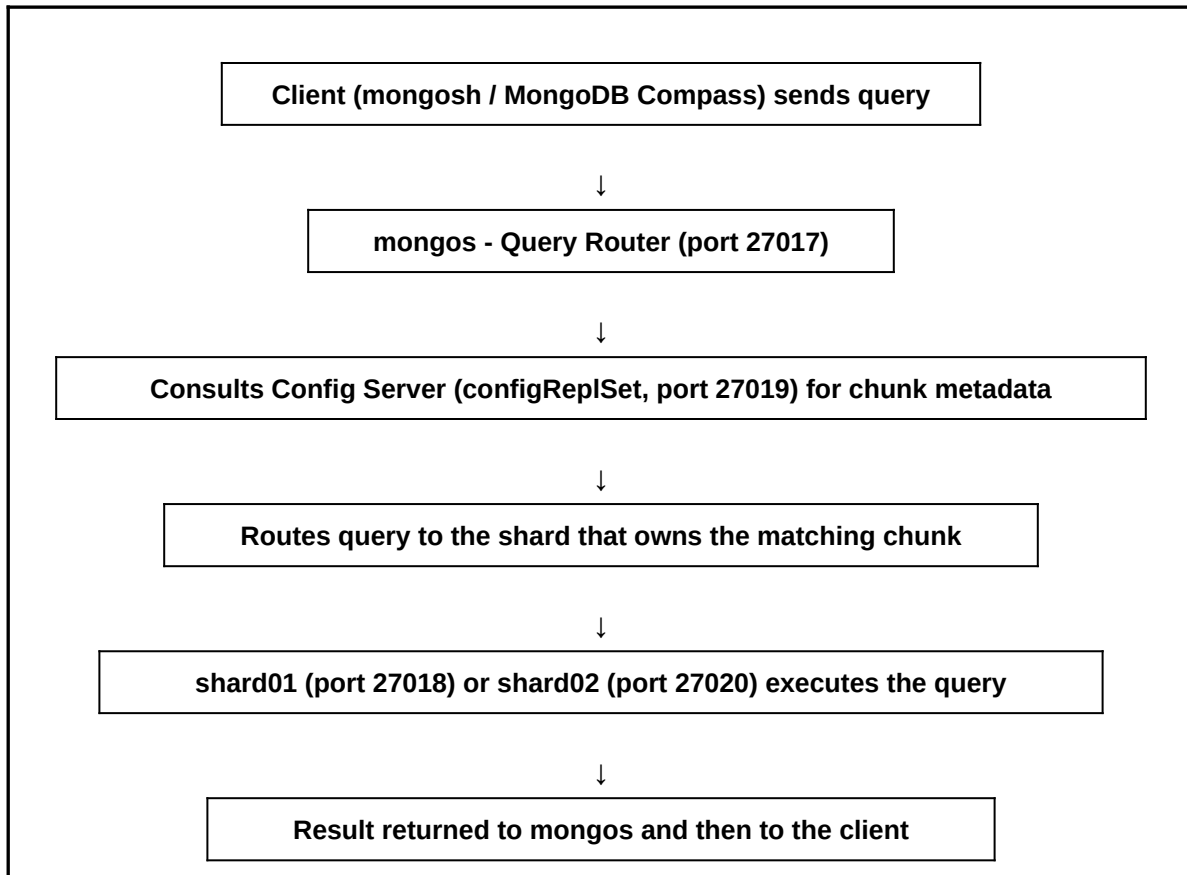
Software / Tool	Version / Image	Purpose
Docker Desktop	Docker Engine + Compose v2	Runs the 4 containers (configsvr, shard01, shard02, mongos) that form the sharded cluster
MongoDB Server	mongo:latest (MongoDB 8.x)	Database engine inside each container; used for config server, shards and router
MongoDB Shell (mongosh)	Included in mongo image	Executes rs.initiate(), sh.addShard(), sh.shardCollection(), insert and query commands
MongoDB Compass	GUI client	Optional GUI used to visually inspect documents through mongos on localhost:27017
VS Code	Latest	Editing docker-compose.yml

		and configuration files
PowerShell / Terminal	Windows	Runs docker commands against the cluster
Git / GitHub	Latest	Version control of the project repository

3) PROCEDURE

Algorithm – Step-by-step procedure

- Step 1: Create a docker-compose.yml file that defines 4 services – configsvr (config server, configReplSet), shard01 (shard1ReplSet), shard02 (shard2ReplSet) and mongos (query router) – on a common bridge network.
- Step 2: Start the containers with “docker compose up -d” and verify all 4 containers are running using “docker ps”.
- Step 3: Initialise the replica sets of the config server and both shards using docker exec with mongosh – rs.initiate() – and confirm each is PRIMARY.
- Step 4: Connect to the query router using “docker exec -it mongos mongosh” and confirm it is a mongos (db.hello() shows msg: 'isdbgrid').
- Step 5: Add both shards to the cluster with sh.addShard("shard1ReplSet/shard01:27017") and sh.addShard("shard2ReplSet/shard02:27017"), then view sh.status().
- Step 6: Enable sharding on the College database (sh.enableSharding("College")) and create the Student collection.
- Step 7: Insert 10 sample student records with RollNo values 1 to 10.
- Step 8: Create an index on RollNo and shard the collection: db.Student.createIndex({ RollNo: 1 }) followed by sh.shardCollection("College.Student", { RollNo: 1 }).
- Step 9: Split the chunk at RollNo: 6 (sh.splitAt) and move the chunk containing RollNo 1 to shard1ReplSet (sh.moveChunk). Verify the distribution with db.Student.getShardDistribution().
- Step 10: Execute the test queries through mongos and observe whether each query is routed to one shard (targeted) or broadcast to all shards (scatter-gather).

Flowchart – Pipeline from input to output**Key configuration – docker-compose.yml**

```
services:
  configsvr:
    image: mongo:latest
    container_name: configsvr
    command: mongod --configsvr --replSet configReplSet --port 27017 --bind_ip_all
    ports: ["27019:27017"]
    volumes: [config_data:/data/db]
    networks: [mongo-cluster]

  shard01:
    image: mongo:latest
    container_name: shard01
    command: mongod --shardsvr --replSet shard1ReplSet --port 27017 --bind_ip_all
    ports: ["27018:27017"]
    volumes: [shard01_data:/data/db]
    networks: [mongo-cluster]

  shard02:
    image: mongo:latest
    container_name: shard02
    command: mongod --shardsvr --replSet shard2ReplSet --port 27017 --bind_ip_all
```

```
ports: ["27020:27017"]
volumes: [shard02_data:/data/db]
networks: [mongo-cluster]

mongos:
  image: mongo:latest
  container_name: mongos
  depends_on: [configsvr, shard01, shard02]
  command: mongos --configdb configReplSet/configsvr:27017 --bind_ip_all --port 27017
  ports: ["27017:27017"]
  networks: [mongo-cluster]

volumes:
  config_data:
  shard01_data:
  shard02_data

networks:
  mongo-cluster:
    driver: bridge
```

Key commands executed

```
docker compose up -d
docker ps

docker exec -it configsvr mongosh --eval "rs.initiate({ _id: 'configReplSet', configsvr: true, members:
[ { _id: 0, host: 'configsvr:27017' } ] })"
docker exec -it shard01 mongosh --eval "rs.initiate({ _id: 'shard1ReplSet', members: [ { _id: 0, host:
'shard01:27017' } ] })"
docker exec -it shard02 mongosh --eval "rs.initiate({ _id: 'shard2ReplSet', members: [ { _id: 0, host:
'shard02:27017' } ] })"

docker exec -it mongos mongosh # connect to the query router
db.hello() # expect: msg: 'isdbgrid'

sh.addShard("shard1ReplSet/shard01:27017")
sh.addShard("shard2ReplSet/shard02:27017")
sh.status()

sh.enableSharding("College")
use College
db.createCollection("Student")
show collections

db.Student.insertMany([ ... 10 student documents with RollNo 1 to 10 ... ])

db.Student.createIndex({ RollNo: 1 })
sh.shardCollection("College.Student", { RollNo: 1 })

sh.splitAt("College.Student", { RollNo: 6 })
sh.moveChunk("College.Student", { RollNo: 1 }, "shard1ReplSet")
db.Student.getShardDistribution()
```

4) TEST CASES – OBSERVATION

The two test queries below were executed inside the mongos shell (docker exec -it mongos mongosh, then use College). The input/output screenshots are also attached in the LMS submission.

Test Case 1 – Targeted query on the shard key (RollNo)

Parameter	Details
Test case	Query a single student using the shard key field
Input	db.Student.find({ RollNo: 3 })
Expected output	mongos reads the chunk map from the Config Server and routes the query only to shard1ReplSet (chunk [MinKey, 6)); returns the single document with RollNo 3
Actual output	One document returned – { RollNo: 3, Name: "Arun", Department: "CSE", Age: 20 } – queried from shard1ReplSet only; shard2 is not involved
Inference	Queries that include the shard key are targeted: only one shard is hit, so the query is fast and uses minimal network hops

Test Case 2 – Scatter-gather query on a non-shard-key field (Department)

Parameter	Details
Test case	Query students using a field that is NOT the shard key
Input	db.Student.find({ Department: "CSE" })
Expected output	mongos broadcasts the query to ALL shards, collects the matching documents from every shard and merges the results
Actual output	5 documents returned (Arun, Bala, Dinesh, Gokul, Karthik) gathered from both shard1ReplSet and shard2ReplSet
Inference	Queries without the shard key are scatter-gather: every shard must be scanned, so such queries are slower than targeted queries

Supporting observation – shard distribution after inserting 10 records and splitting at RollNo 6:

Shard	Documents	RollNo Range
shard1ReplSet	5	1 to 5
shard2ReplSet	5	6 to 10

Output screenshots (also uploaded on LMS)

Output 1 – All four Docker containers (configsvr, shard01, shard02, mongos) running

```

Microsoft Windows [Version 10.0.26200.8875]
(c) Microsoft Corporation. All rights reserved.

C:\Users\HP>
C:\Users\HP>
C:\Users\HP>docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
54cfa6b447e5   mongo:latest   "docker-entrypoint.s..." 19 minutes ago Up 19 minutes 0.0.0.0:27017->27017/tcp, [::]:27017->27017/tcp mongos
aa2c47384567   mongo:latest   "docker-entrypoint.s..." 19 minutes ago Up 19 minutes 0.0.0.0:27019->27017/tcp, [::]:27019->27017/tcp configsvr
c65fc9fe24ba   mongo:latest   "docker-entrypoint.s..." 19 minutes ago Up 19 minutes 0.0.0.0:27018->27017/tcp, [::]:27018->27017/tcp shard01
df92a85517fe   mongo:latest   "docker-entrypoint.s..." 19 minutes ago Up 19 minutes 0.0.0.0:27020->27017/tcp, [::]:27020->27017/tcp shard02

C:\Users\HP>

```

Output 2 – Config server replica set initialised and PRIMARY

```

Command Prompt x Command Prompt x Command Prompt x Command Prompt x mongosh mongoc Command Prompt x Command Prompt x
newTermStartDate: ISODate('2026-08-12T09:44:56.974Z'),
wMajorityWriteAvailabilityDate: ISODate('2026-08-12T09:44:57.664Z')
},
members: [
  {
    _id: 0,
    name: 'configsvr:27017',
    health: 1,
    state: 1,
    stateStr: 'PRIMARY',
    uptime: 1223,
    optime: { ts: Timestamp({ t: 1786529052, i: 1 }), t: Long('1') },
    optimeDate: ISODate('2026-08-12T10:04:12.000Z'),
    optimeWritten: { ts: Timestamp({ t: 1786529052, i: 1 }), t: Long('1') },
    optimeWrittenDate: ISODate('2026-08-12T10:04:12.000Z'),
    lastAppliedWallTime: ISODate('2026-08-12T10:04:12.646Z'),
    lastDurableWallTime: ISODate('2026-08-12T10:04:11.645Z'),
    lastWrittenWallTime: ISODate('2026-08-12T10:04:12.646Z'),
    syncSourceHost: '',
    syncSourceId: -1,
    infoMessage: '',
    electionTime: Timestamp({ t: 1786527896, i: 2 }),
    electionDate: ISODate('2026-08-12T09:44:56.000Z'),
    configVersion: 1,
    configTerm: 1,
    self: true,
    lastHeartbeatMessage: ''
  }
],
ok: 1,
'$clusterTime': {
  clusterTime: Timestamp({ t: 1786529052, i: 1 }),
  signature: {
    hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
    keyId: Long('0')
  }
},
operationTime: Timestamp({ t: 1786529052, i: 1 })
}
C:\Users\HP>

```

Output 3 – Shard 1 replica set initialised and PRIMARY

```

Command Prompt x Command Prompt x Command Prompt x Command Prompt x mongosh mongoc Command Prompt x Command Prompt x
newTermStartDate: ISODate('2026-08-12T09:45:12.587Z'),
wMajorityWriteAvailabilityDate: ISODate('2026-08-12T09:45:12.726Z')
},
members: [
  {
    _id: 0,
    name: 'shard01:27017',
    health: 1,
    state: 1,
    stateStr: 'PRIMARY',
    uptime: 1404,
    optime: { ts: Timestamp({ t: 1786529230, i: 1 }), t: Long('1') },
    optimeDate: ISODate('2026-08-12T10:07:10.000Z'),
    optimeWritten: { ts: Timestamp({ t: 1786529230, i: 1 }), t: Long('1') },
    optimeWrittenDate: ISODate('2026-08-12T10:07:10.000Z'),
    lastAppliedWallTime: ISODate('2026-08-12T10:07:10.513Z'),
    lastDurableWallTime: ISODate('2026-08-12T10:07:10.513Z'),
    lastWrittenWallTime: ISODate('2026-08-12T10:07:10.513Z'),
    syncSourceHost: '',
    syncSourceId: -1,
    infoMessage: '',
    electionTime: Timestamp({ t: 1786527912, i: 2 }),
    electionDate: ISODate('2026-08-12T09:45:12.000Z'),
    configVersion: 1,
    configTerm: 1,
    self: true,
    lastHeartbeatMessage: ''
  }
],
ok: 1,
'$clusterTime': {
  clusterTime: Timestamp({ t: 1786529230, i: 1 }),
  signature: {
    hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
    keyId: Long('0')
  }
},
operationTime: Timestamp({ t: 1786529230, i: 1 })
}
C:\Users\HP>

```

Output 4 – Shard 2 replica set initialised and PRIMARY

```

},
members: [
  {
    _id: 0,
    name: 'shard02:27017',
    health: 1,
    state: 1,
    stateStr: 'PRIMARY',
    uptime: 1479,
    optime: { ts: Timestamp({ t: 1786529301, i: 1 }), t: Long('1') },
    optimeDate: ISODate('2026-08-12T10:08:21.000Z'),
    optimeWritten: { ts: Timestamp({ t: 1786529301, i: 1 }), t: Long('1') },
    optimeWrittenDate: ISODate('2026-08-12T10:08:21.000Z'),
    lastAppliedWallTime: ISODate('2026-08-12T10:08:21.872Z'),
    lastDurableWallTime: ISODate('2026-08-12T10:08:21.872Z'),
    lastWrittenWallTime: ISODate('2026-08-12T10:08:21.872Z'),
    syncSourceHost: '',
    syncSourceId: -1,
    infoMessage: '',
    electionTime: Timestamp({ t: 1786527919, i: 1 }),
    electionDate: ISODate('2026-08-12T09:45:19.000Z'),
    configVersion: 1,
    configTerm: 1,
    self: true,
    lastHeartbeatMessage: ''
  }
],
ok: 1,
'scClusterTime': {
  clusterTime: Timestamp({ t: 1786529304, i: 1 }),
  signature: {
    hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAAA', 0),
    keyId: Long('0')
  }
},
operationTime: Timestamp({ t: 1786529301, i: 1 })
}
C:\Users\HP>
C:\Users\HP>

```

Output 5 – mongos query router verified (db.hello()) shows msg: 'isdbgrid'

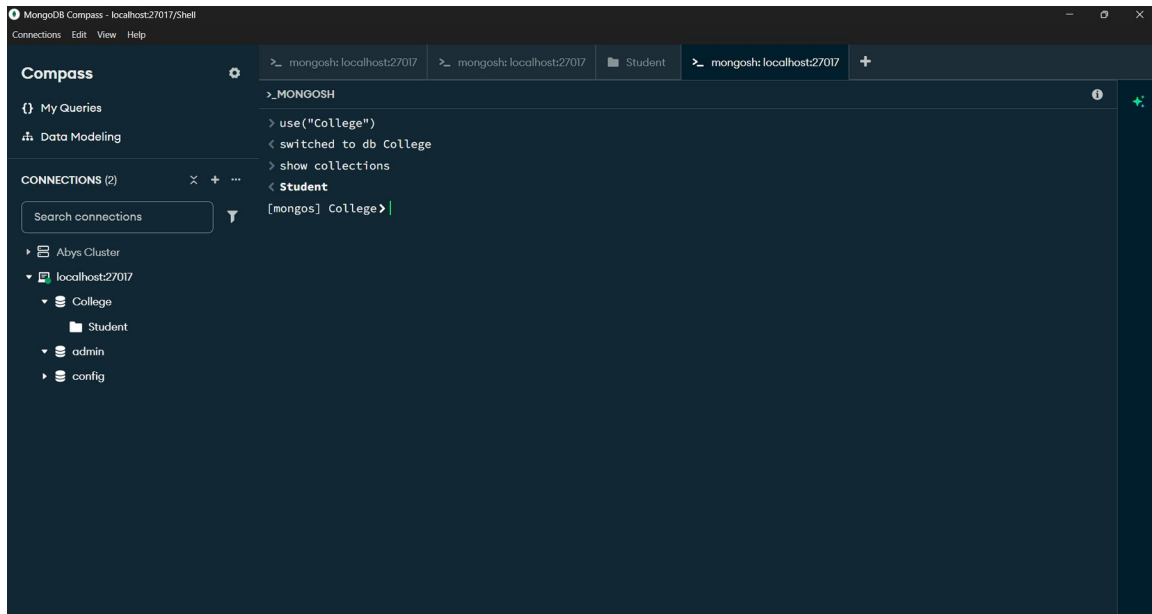
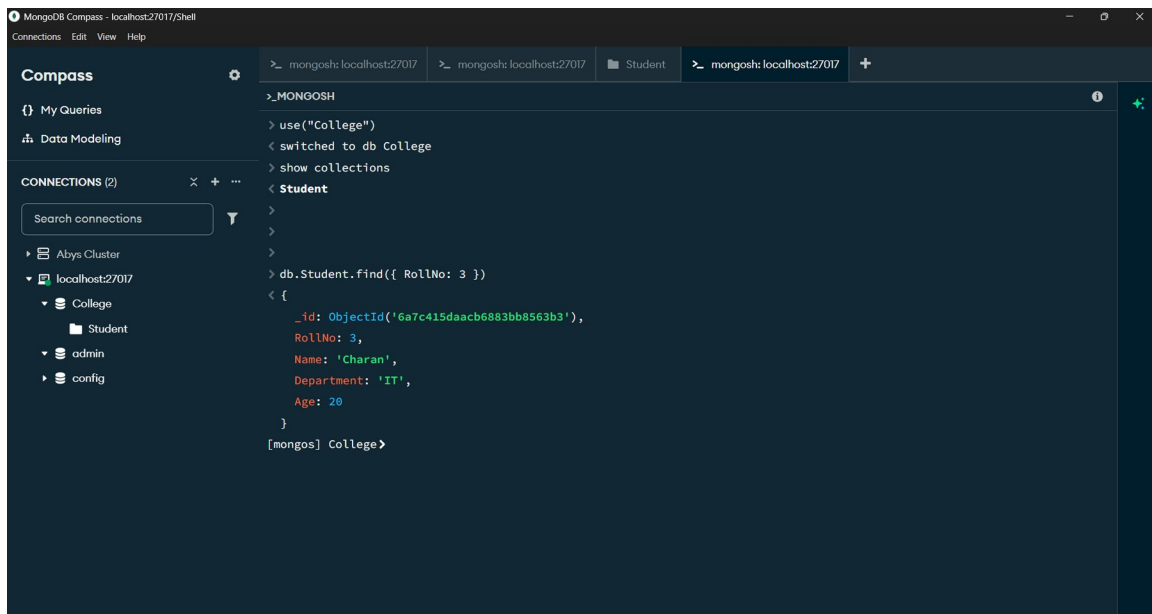
```

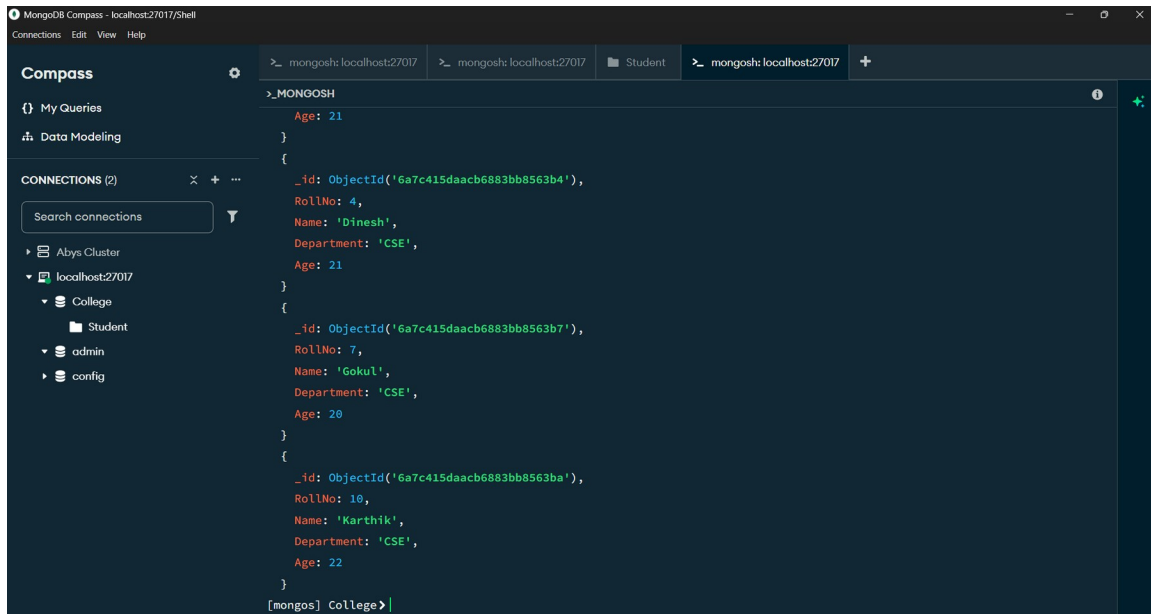
MongoDB Compass - localhost:27017/Shell
Connections Edit View Help

> mongosh: localhost:27017
> mongosh: localhost:27017
Student

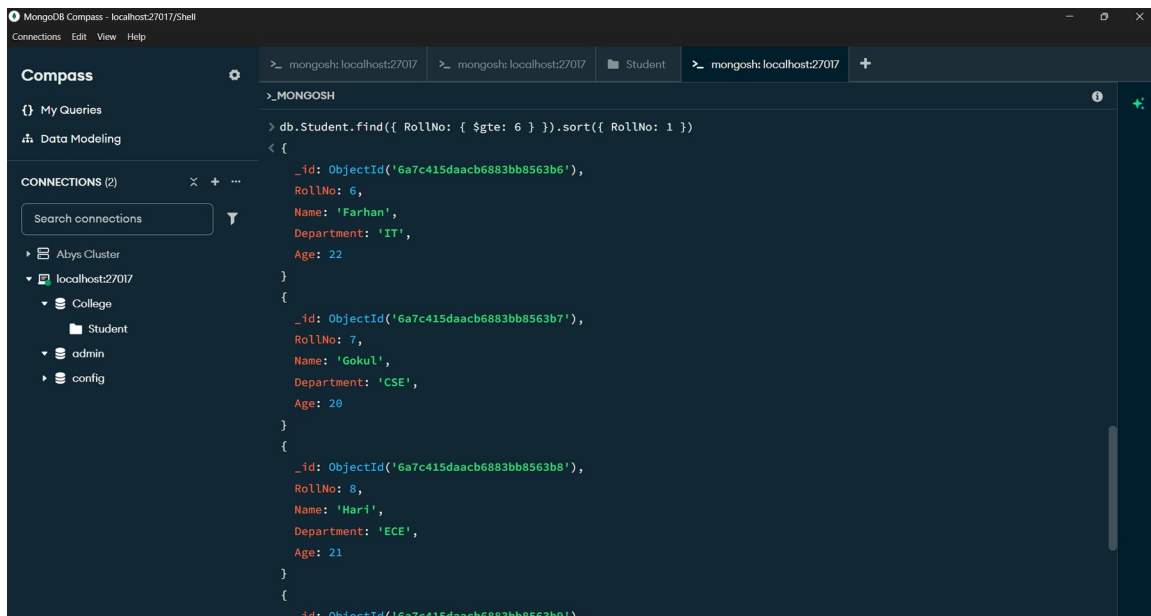
>_MONGOSH
< switched to db College
> db.hello()
< {
  isWritablePrimary: true,
  msg: 'isdbgrid',
  topologyVersion: { processId: ObjectId('6a7c4057f27ee86c17aee9c4'), counter: Long('0') },
  maxBsonObjectSize: 16777216,
  maxMessageSizeBytes: 48000000,
  maxWriteBatchSize: 100000,
  localTime: 2026-08-12T10:06:13.090Z,
  logicalSessionTimeoutMinutes: 30,
  connectionId: 67,
  maxWireVersion: 27,
  minWireVersion: 0,
  ok: 1,
  'scClusterTime': {
    clusterTime: Timestamp({ t: 1786529171, i: 1 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAAA', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1786529171, i: 1 })
}

```

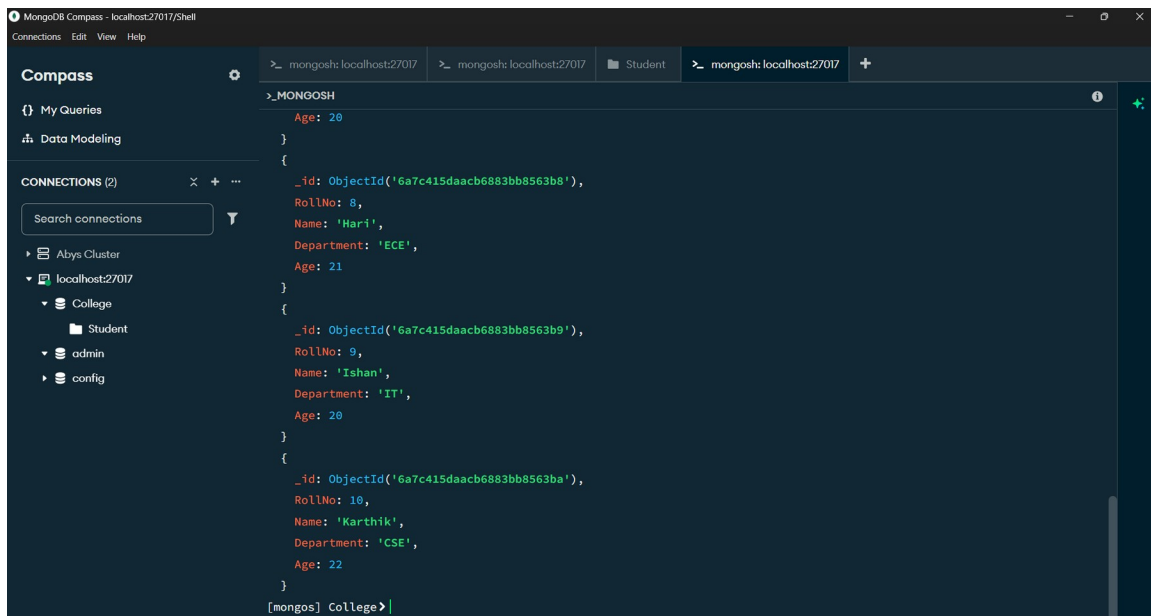

Output 6 – Student collection created in the College database**Output 7 – Test Case 1: targeted query on shard key (RollNo)**

Output 8 – Test Case 2: scatter-gather query on non-shard key (Department)

```
>_MONGOSH
Age: 21
}
{
  _id: ObjectId('6a7c415daacb6883bb8563b4'),
  RollNo: 4,
  Name: 'Dinesh',
  Department: 'CSE',
  Age: 21
}
{
  _id: ObjectId('6a7c415daacb6883bb8563b7'),
  RollNo: 7,
  Name: 'Gokul',
  Department: 'CSE',
  Age: 20
}
{
  _id: ObjectId('6a7c415daacb6883bb8563ba'),
  RollNo: 10,
  Name: 'Karthik',
  Department: 'CSE',
  Age: 22
}
[mongos] College>
```

Output 9 – Range query on shard key, RollNo >= 6 (part 1)

```
>_MONGOSH
> db.Student.find({ RollNo: { $gte: 6 } }).sort({ RollNo: 1 })
< {
  _id: ObjectId('6a7c415daacb6883bb8563b6'),
  RollNo: 6,
  Name: 'Farhan',
  Department: 'IT',
  Age: 22
}
{
  _id: ObjectId('6a7c415daacb6883bb8563b7'),
  RollNo: 7,
  Name: 'Gokul',
  Department: 'CSE',
  Age: 20
}
{
  _id: ObjectId('6a7c415daacb6883bb8563b8'),
  RollNo: 8,
  Name: 'Harid',
  Department: 'ECE',
  Age: 21
}
{
  _id: ObjectId('6a7c415daacb6883bb8563b9'),
```

Output 10 – Range query on shard key, RollNo >= 6 (part 2)


```

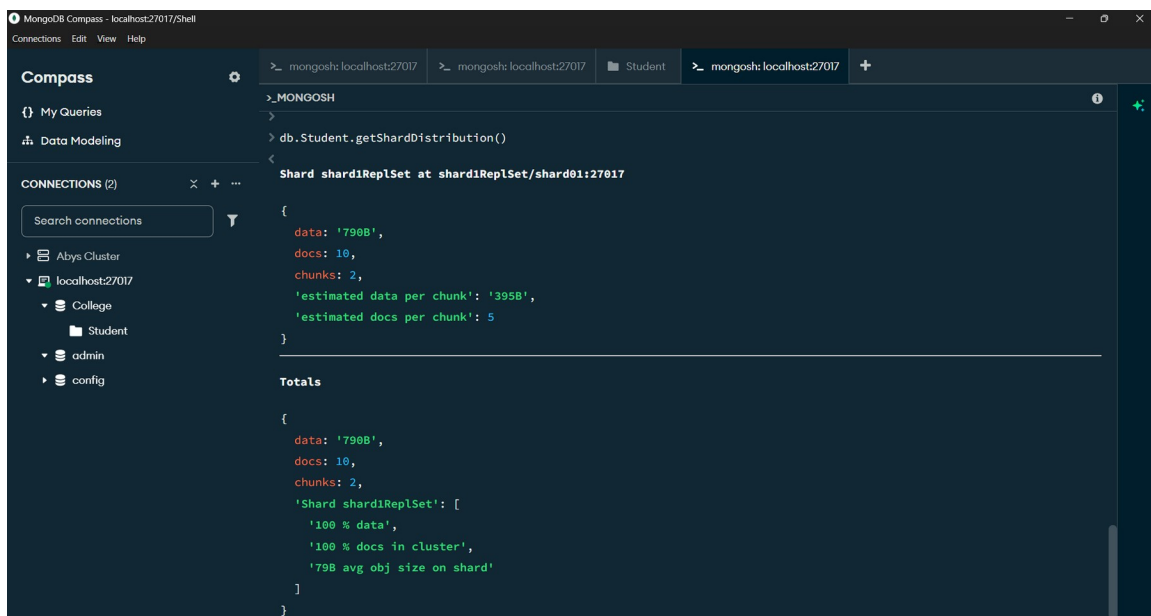
MongoDB Compass - localhost:27017/Shell
Connections Edit View Help

Compass
My Queries
Data Modeling

CONNECTIONS (2)
Search connections
Abys Cluster
localhost:27017
  College
    Student
  admin
  config

>_ MONGOSH
Age: 20
}
{
  _id: ObjectId('6a7c415daacb6883bb8563b8'),
  RollNo: 8,
  Name: 'Hari',
  Department: 'ECE',
  Age: 21
}
{
  _id: ObjectId('6a7c415daacb6883bb8563b9'),
  RollNo: 9,
  Name: 'Ishan',
  Department: 'IT',
  Age: 20
}
{
  _id: ObjectId('6a7c415daacb6883bb8563ba'),
  RollNo: 10,
  Name: 'Karthik',
  Department: 'CSE',
  Age: 22
}
[mongos] College>

```

Output 11 – Shard distribution showing 5 documents per shard


```

MongoDB Compass - localhost:27017/Shell
Connections Edit View Help

Compass
My Queries
Data Modeling

CONNECTIONS (2)
Search connections
Abys Cluster
localhost:27017
  College
    Student
  admin
  config

>_ MONGOSH
>
> db.Student.getShardDistribution()
<
Shard shard1ReplSet at shard1ReplSet/shard01:27017

{
  data: '790B',
  docs: 10,
  chunks: 2,
  'estimated data per chunk': '395B',
  'estimated docs per chunk': 5
}

Totals

{
  data: '790B',
  docs: 10,
  chunks: 2,
  'Shard shard1ReplSet': [
    '100 % data',
    '100 % docs in cluster',
    '790B avg obj size on shard'
  ]
}

```

5) LEARNING OUTCOMES

- Installed and configured Docker Desktop and created a multi-container application using Docker Compose.
- Deployed a MongoDB sharded cluster consisting of config servers, shard servers and a mongos query router.
- Configured replica sets and shard servers and understood their role in high availability.
- Understood the architecture of MongoDB sharding (shards, chunks, config server, balancer).
- Configured horizontal sharding using an appropriate shard key (RollNo) and justified the choice.
- Analyzed how documents are distributed across shards and how queries are routed (targeted vs scatter-gather).
- Explained the scalability benefits of a sharded database over a single MongoDB instance.